
THE EVOLUTION OF MACHINE LEARNING AND AI

Week 3 Core Booklet

How Machines Learn: the required reading, in plain English

Module	The Evolution of Machine Learning and AI (5 ECTS, Semester 1)
Week	Week 3 of 12
Learning outcomes	LO2: Characterise the training of the main families of model · LO4: Identify the tools and frameworks used in contemporary AI and ML
Assessed in	Week 6 quiz (50% of the module mark). Everything the quiz can ask about Week 3 is in this booklet.
Reading time	About 45 minutes. Read it once, slowly, after the lecture and before the lab.
Goes with	The Week 3 lecture · the six-exercise lab, one exercise per chapter · SUBMISSION.md

What this booklet is. It is the **required** reading for Week 3, and it is short on purpose. It has the same six parts as the lecture, in the same order, using the same words. Every red ★ **For the Quiz** box is a statement the Week 6 quiz can test, written the way you should be able to say it. Grey **Extra** boxes tell you where an optional, deeper account lives in the **full** booklet; you are not required to read the full booklet at all. There is no maths here beyond add, subtract, multiply, and there are exactly two formulas, both explained in words first.

Contents

1	What “learning” means	3
2	Four shapes of model	5
3	Memorising is not learning	7
4	Walking downhill: gradient descent	9
5	One neuron, its wall, and the hidden layer	11
6	Blame, and the five lines of PyTorch	13
7	The whole week on one page	15
8	Twenty practice questions, with answers	17

YOUR WEEK, IN ORDER

1	The lecture (or its recording).	2 h
2	This booklet. Once, slowly.	45 min
3	The lab: six exercises, one per chapter of this booklet, in the same order.	6 × 20 min
4	Check SUBMISSION.md (already filled in) against your run; upload it.	10 min
+	Optional: the “going further” section of the lab, and the full booklet.	only if you want

That is about five hours. If an exercise takes more than 40 minutes, stop and ask in the channel.

1 What “learning” means

COVERS Lecture Part 1 · Lab Exercise 1

TIME 7 minutes

IN BRIEF A model is a formula with dials. Learning means turning the dials until the guesses are close to the real answers. The computer does the turning, in a four-step loop.

1.1 A formula with dials

Suppose you want to guess a house’s price from its size. A very simple model is

$$\text{price} = a \times \text{size} + b$$

The letters **a** and **b** are the **dials**. Set them badly and every guess is wrong. Set them well and the guesses are close to the real prices. The formula is the **model**; the dials are what gets **learned**.

IN PLAIN ENGLISH

Learning is turning the dials until the guesses match the answers. The computer turns them. That is all “training a model” means. Everything else in this booklet is about **how** it turns them and **what shape of formula** to use.

1.2 The loop: guess, check, blame, nudge

The computer turns the dials by going round a four-step loop, thousands of times:

Step	What happens	What the code calls it
1 Guess	Put the input through the formula with the dials as they are now and get a guess.	the forward pass
2 Check	Compare the guess to the real answer. Turn the difference into one number: big number = very wrong.	the loss
3 Blame	Work out which dials caused the mistake, and how much each one is to blame.	the backward pass , or backpropagation
4 Nudge	Turn every dial a tiny bit in the direction that makes the loss smaller.	the update , or optimiser step

Then go round again. Every model in this module, from a straight line to a chatbot, is this loop with a different formula in step 1.

EVERYDAY ANALOGY

The hillwalker in fog. You are on a hillside in thick fog trying to reach the valley floor. You cannot see the valley. But you can feel the slope under your feet, so you step downhill, feel again, and step again. Height = how wrong the model is. Your position = the dial settings. A step = one nudge. The valley floor = the dials that make the model least wrong. Every new word in this booklet is a fact about that walk.

1.3 Two words you will hear every week

- **Parameters** are the dials the **computer** turns: the **a** and **b** above. A big language model has billions; the network in this week’s lab has nine.
- **Hyperparameters** are the settings **you** choose before training starts: how big a step to take, how many layers, how deep a tree. The computer never chooses these.

1.4 Three choices you make before pressing “train”

1. **What shape of answer?** A straight line? A set of yes/no questions? A wiggly curve? (Part 2.)
2. **How do we score a wrong answer?** Usually “how far off was the guess, squared”. This score is the **loss**. You pick it from a menu; you never invent one.
3. **How do we search for good dial settings?** Almost always: walk downhill one small step at a time. (Part 4.)

When a textbook says “we minimise a loss function over a hypothesis class using an optimiser”, it means exactly these three choices.

★ FOR THE QUIZ

- A model is a formula with **dials** (parameters). Learning = turning the dials so the guesses match the answers.
- The training loop is **Guess → Check → Blame → Nudge**, repeated many times. Technical names: forward pass, loss, backward pass, update.
- **Parameters** are learned by the computer. **Hyperparameters** are chosen by you.
- Before training you choose: the **shape** of the answer, how to **score** a mistake, and how to **search**.

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapters 1–3: the formal statements, the notation $(\theta, L, \hat{y})$, supervised vs unsupervised learning, and how to read a formula out loud.

2 Four shapes of model

COVERS Lecture Part 2 · Lab Exercise 2

TIME 10 minutes

IN BRIEF Choosing a model is choosing a **shape** of answer. Four shapes cover most of what you will meet: straight lines, right-angled steps, the widest corridor, and many small bends.

Picture dots of two colours on a page. Your model's job is to draw a boundary between the colours. Different models can only draw different **shapes** of boundary, and that shape is the model's built-in assumption about the world.

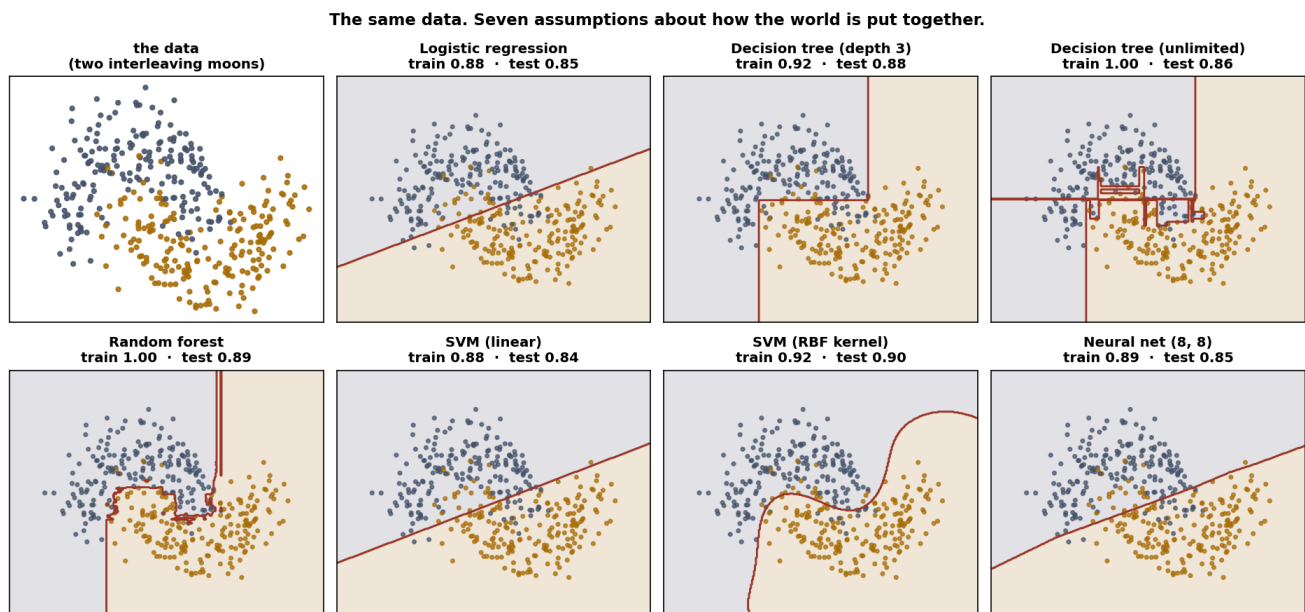


Figure 1: Seven models trained on the same dots (from the lab). Top row, left to right: logistic regression, a small tree, an unlimited tree, a random forest. Bottom row: a linear SVM, an RBF SVM, a small neural network. Look only at the **shape** each one draws.

2.1 Shape 1 · the straight line: linear and logistic regression

Linear regression predicts a **number** with a straight line; the house-price formula is one. **Logistic regression** is the same straight line with its answer squashed to sit between 0 and 1, so you can read it as a probability: “70% chance this customer leaves”. Same line, two jobs. It is the right **first** thing to try on most real problems: fast, stable, and every dial is a sentence (“each extra square metre adds €2,000”).

2.2 Shape 2 · right-angled steps: the decision tree

A tree plays twenty questions, one fact at a time: “Is income above €32,000? If yes, is tenure above 14 months?” Each question splits the page with a vertical or horizontal line, so a tree can only ever draw boundaries made of right-angled steps. You can read its rules out loud, which is a real virtue. The price: it keeps asking questions until every box holds a single dot, which is the **memorising** problem of Part 3. (A **random forest** averages hundreds of slightly different trees; it is more accurate and less readable. Extra.)

2.3 Shape 3 • the widest corridor: the support vector machine (SVM)

If a straight line can separate the colours, there are usually many such lines. The SVM picks the one that leaves the **widest empty corridor** between the two colours, on the reasoning that a boundary with room to spare copes better with new dots. If no straight line works, the SVM has a trick for bending the page so that one does; that is the **kernel trick**, and it is extra.

2.4 Shape 4 • many small bends: the neural network

A neural network is a stack of very simple units, each doing **multiply, add, then bend**. Stack enough of them and they can draw almost any curve. It is the most flexible of the four, needs the most data, and is the hardest to explain afterwards. Part 5 shows the smallest one there is.

IN PLAIN ENGLISH

Choosing a model is choosing an assumption about the shape of the answer. A straight-line model assumes the answer is a straight line; if it is not, no amount of data will fix that. Week 2 called this the **good** sense of the word “bias”: without some assumption, there is no basis for choosing one answer over the infinitely many others that fit the same data.

★ FOR THE QUIZ

- Four shapes: **straight line** (linear / logistic regression), **right-angled steps** (decision tree), **widest corridor** (SVM), **many small bends** (neural network).
- **Choosing a model is choosing an assumption** about the shape of the answer.
- Trees are readable but memorise. Lines are simple and stable. Networks are flexible but need data and are hard to explain.

EXTRA • OPTIONAL • NOT ON THE QUIZ

Full booklet Chapters 4–7: the equations of each model, Gini impurity, random forests, soft margins, and the kernel trick.

3 Memorising is not learning

COVERS Lecture Part 3 · Lab Exercise 3

TIME 6 minutes

IN BRIEF A model can be too simple (underfitting) or memorise the examples (overfitting). You spot the second by the **gap** between its score on data it studied and data it has never seen.

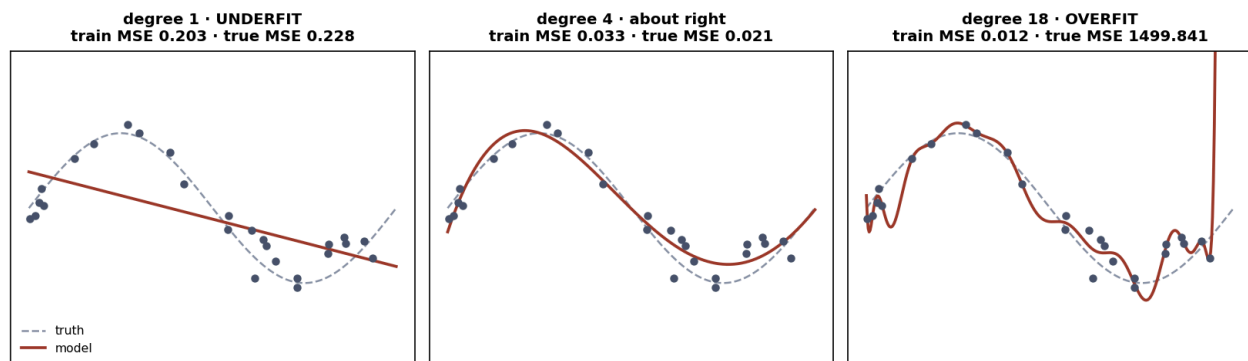


Figure 2: The same twelve dots fitted three times (from the lab). Left: a straight line, too simple. Middle: about right. Right: a very wiggly curve that passes through every dot exactly and is hopeless in between.

- **Underfitting:** the model is too simple to capture the pattern. It does badly on the training dots **and** on new dots. The straight line through a wave.
- **Overfitting:** the model has memorised the training dots, noise and all. It does brilliantly on the training dots and badly on new dots. The wiggly curve. In the lab it has the lowest training error of the three and a true error about **seventy thousand times** worse than the middle one.

3.1 How you would know: the gap

Split your data into dots the model **studies** (the **training set**) and dots it **never sees** until the end (the **test set**). Score it on both. Then read the **gap**:

model	train	test	gap
Decision tree (depth 3)	0.915	0.876	0.039
Decision tree (unlimited)	1.000	0.857	0.143

The unlimited tree got **every** training dot right and is beaten on new dots by a tree a fraction of its size. A perfect training score is not an achievement; it is the **symptom** of memorising.

EVERYDAY ANALOGY

A student who memorises last year's exam paper scores 100% on last year's paper and falls apart on this year's. You would not conclude they had learned the subject. You would look at the gap.

COMMON MISTAKE

Reporting a model's accuracy on the data it was trained on. That number is close to meaningless. Always report the score on data the model has never seen, and always look at the gap between the two.

★ FOR THE QUIZ

- **Underfitting:** too simple; bad on training data and new data. **Overfitting:** memorised the training data; great on training data, bad on new data.
- You spot overfitting by the **gap** between the training score and the test score. Never report training accuracy on its own.
- The **test set** must be data the model has never seen. That is its entire purpose.

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapter 8: the bias–variance decomposition (the dartboard picture), why random forests memorise and still generalise, and regularisation.

4 Walking downhill: gradient descent

COVERS Lecture Part 4 · Lab Exercise 4

TIME 9 minutes

IN BRIEF How the dials get turned: feel the slope, take a small step down, repeat. The size of the step is the **learning rate**, and it is the one setting that can silently break everything.

4.1 The method, in words

Take the hillwalker seriously. At the current dial settings, the computer works out the **slope**: if I turn this dial up a little, does the loss go up or down, and how steeply? Then it turns the dial a small step in the **downhill** direction. Then it does it again. Near the bottom the slope is gentle, so the steps get smaller on their own; nobody has to tell it to slow down. That is **gradient descent** (“gradient” is just the maths word for slope).

4.2 The one formula

$$\text{new dial} = \text{old dial} - (\text{step size}) \times (\text{slope})$$

The minus sign means “go down, not up”. In symbols it is written $\theta \leftarrow \theta - \eta \cdot \frac{\partial L}{\partial \theta}$, where θ is the dial, η is the step size and $\partial L / \partial \theta$ is the slope. You will see it written that way in books; now you can read it.

4.3 The step size is the whole ballgame

The step size is called the **learning rate**, and it is a hyperparameter: you choose it. The lab tries five values on the same simple bowl-shaped hill, starting from the same point:

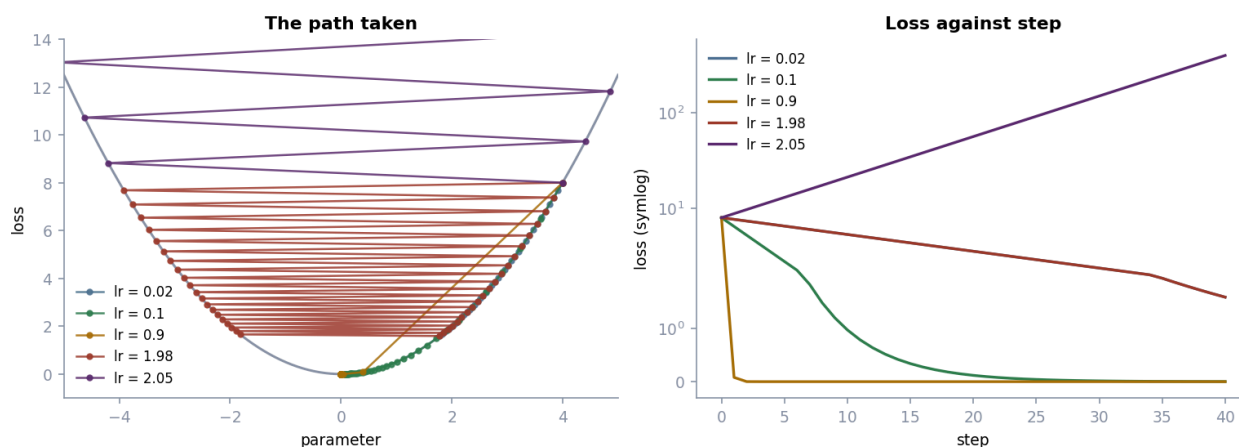


Figure 3: Five learning rates on the same bowl (from the lab). Left: the path each walker took. Right: the loss at each step.

learning rate	what happened	final loss
0.02	Too small. Right direction, tiny steps. Still walking when time ran out.	1.59
0.10	About right. Straight down.	0.002
0.90	Also fine. Straight down, faster.	0.000
1.98	Too big. Every step jumps over the bottom to the other side. It bounces and barely gets lower.	1.59
2.05	Far too big. Each bounce is bigger than the last. Off to infinity.	396 and rising

Two things to notice. First, the **good band is narrow** and nothing tells you in advance where it is; you find it by trying. Second, when it explodes, Python prints **no error**. The loss just becomes nan (not a number) and the program carries on. That silence is why the lecture called the learning rate dangerous.

4.4 One honest complication: not every hill is a bowl

On a bumpy hill the walker can settle in a **small dip** that is not the valley floor. It feels flat underfoot, so it stops. That is a **local minimum**. In the lab, three walkers start at different points with the same step size; two reach the good valley and the one starting at 5.5 stops in a worse dip. Where you start can decide where you end.

★ FOR THE QUIZ

- **Gradient descent**: feel the slope, take a small step downhill, repeat. Steps shrink on their own near the bottom.
- The formula: **new dial = old dial – step size × slope**.
- The step size is the **learning rate**. Too small: crawls. Too big: bounces. Far too big: explodes to nan with no error message.
- A **local minimum** is a small dip that is not the true bottom; the walker can get stuck there.

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapters 9–10: batch vs mini-batch vs stochastic descent, momentum, saddle points, and why local minima matter less in very high dimensions.

5 One neuron, its wall, and the hidden layer

COVERS Lecture Part 5 · Lab Exercise 5

TIME 9 minutes

IN BRIEF One artificial neuron (1958) can only draw a straight line, so it fails on four dots called XOR (1969). One hidden layer fixes it by **moving the dots** (1986). This is Week 1's story, seen from the inside.

5.1 One artificial neuron: the perceptron (1958)

Take the inputs. Multiply each by a dial (called a **weight**). Add them up. If the total is above a threshold, answer **yes**; otherwise **no**. That is the whole model:

$$\hat{y} = \begin{cases} 1 & \text{if } w_1x_1 + w_2x_2 + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

It is the house formula with a yes/no bolted on. It learns by the loop: guess, check, and if wrong, nudge the weights toward the right answer. Rosenblatt proved it will always succeed, **provided the two colours can be separated by a straight line**. Everything that happened next lives in that “provided”.

5.2 The wall: XOR (1969)

Four dots. The rule: output 1 when the two inputs are **different**, 0 when they are the **same**.

input 1	input 2	answer
0	0	0
0	1	1
1	0	1
1	1	0

Draw them on a page and try to put one straight line with both “1” dots on one side and both “0” dots on the other. You cannot; nobody can. There is no such line, so the perceptron can never learn XOR, however long it trains. In the lab it never gets more than 3 of the 4 right.

5.3 The way through: a hidden layer moves the dots (1986)

Put two neurons **in between** the input and the answer: a **hidden layer**. Now plot what those two neurons output for each of the four dots. In the lab the numbers come out like this:

input	what the hidden layer turns it into	answer
(0, 0)	(0.00, 0.91)	0
(0, 1)	(0.08, 0.02)	1
(1, 0)	(0.08, 0.02)	1
(1, 1)	(0.87, 0.00)	0

The two dots whose answer is 1 have been moved **on top of each other**. Now one straight line separates the 1s from the 0s, and the last neuron draws that line. 4 of 4 correct.

IN PLAIN ENGLISH

A **hidden layer moves the dots so that a straight line works**. That is the whole idea. What took the field seventeen years was not the idea but learning how to **train** the hidden layer, which is Part 6.

5.4 Why each neuron needs a “bend”

A straight line followed by another straight line is still a straight line. Stack a hundred straight-line layers and you still cannot draw XOR. So after each neuron adds things up, the total goes through a **bend**: a squashing function, called an **activation function**. The classic one, the **sigmoid**, squeezes any number into the range 0 to 1. The modern one, **ReLU**, simply replaces negative numbers with zero. No bend, no curves. In the lab’s browser Playground, switch the bend off and watch XOR fail again.

★ FOR THE QUIZ

- A **perceptron** (1958) is one neuron: multiply inputs by weights, add, then yes/no. It can only draw a straight line.
- **XOR** (1969): four dots that no straight line separates. The perceptron can never learn it.
- A **hidden layer** moves the dots into a new arrangement where a straight line **does** work.
- Each neuron needs a **bend** (activation function: sigmoid, ReLU). Without it, many layers collapse into one straight line.

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapters 11 and 14: Rosenblatt’s convergence theorem, the three activation functions drawn, and the vanishing-gradient problem that made ReLU win.

6 Blame, and the five lines of PyTorch

COVERS Lecture Part 6 · Lab Exercise 6

TIME 7 minutes

IN BRIEF The “Blame” step has a name, backpropagation. You need one sentence about it. Then: why deep learning waited until 2012, and the five lines of PyTorch you will copy for the rest of the module.

6.1 Backpropagation, in one sentence

The output was wrong by some amount. There are nine dials. **How much of the mistake is each dial’s fault?** For the dials right next to the output that is easy. For a dial two layers back, its effect was filtered through everything in front of it.

The answer is to walk **backwards**: start at the loss, ask the output layer how much it contributed, pass a share of the blame back to the hidden layer, and pass a share of **that** back to the input weights. Every dial ends up knowing which way to turn.

★ FOR THE QUIZ

Backpropagation is the method for working out how much each weight is to blame for the loss, starting from the output and working backwards. It is the “Blame” step. It does not do the learning; gradient descent then uses the blame to nudge the weights.

You do not need to be able to do this by hand. Underneath, it is the chain rule from calculus applied over and over; the lab’s `scratch_net.py` writes it out in about ten lines so you can **see** it, and PyTorch does the whole thing in one line, `loss.backward()`.

6.2 Why it waited until 2012

Backpropagation was published in 1986 and the maths barely changed for 26 years. Deep learning took off in 2012 because the **ingredients** arrived, not a new recipe:

- **Data (2009).** ImageNet: 14 million labelled pictures. The loop needs examples.
- **Compute (2007).** Graphics cards (GPUs) turned out to be perfect for millions of small multiply-and-adds.
- **Small fixes.** A better bend (ReLU), better starting values for the dials, a few tricks.

AlexNet in 2012 used 1986’s algorithm on 2009’s data with 2007’s hardware.

6.3 The five lines

Here is the training loop in PyTorch, the framework you will use from Week 4 on. Each line is one step of the loop from Part 1:

```
for epoch in range(epochs):
    optimiser.zero_grad()      # 0. wipe last time's blame
    pred = model(X)            # 1. GUESS    (forward)
    loss = loss_fn(pred, Y)     # 2. CHECK    (loss)
    loss.backward()             # 3. BLAME    (backward)
    optimiser.step()            # 4. NUDGE    (update)
```

Line 0 is housekeeping: PyTorch remembers last time's blame unless you wipe it. Forget line 0 and the code **still runs**, just wrongly. You will paste these five lines into every lab from here to Week 12, and nothing in them is magic: you have now seen what each one does.

★ FOR THE QUIZ

- Deep learning took off in **2012** because **data** (ImageNet) and **compute** (GPUs) finally arrived for a 1986 algorithm.
- PyTorch's training loop is five lines: **zero_grad, forward, loss, backward, step**, matching the loop: wipe, guess, check, blame, nudge.
- `loss.backward()` computes the blame (the gradients). It does not “learn”; `optimiser.step()` does the nudging.

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapters 15–17: the chain rule written out with real numbers, the gradient check, the `zero_grad` trap in detail, and what a framework does for you.

7 The whole week on one page

COVERS Revision
TIME 3 minutes to read; 10 to memorise
IN BRIEF Every quiz-able statement from Week 3, in one place.

Topic	The sentence to be able to say
The loop	Guess → Check → Blame → Nudge, repeated. (Forward, loss, backward, update.)
Parameters	The dials the computer learns. Hyperparameters : the settings you choose (learning rate, layer sizes, tree depth).
Three choices	Shape of the answer · how to score a mistake (loss) · how to search (gradient descent).
Four shapes	Straight line (linear/logistic regression) · right-angled steps (tree) · widest corridor (SVM) · many small bends (neural net).
Assumption	Choosing a model is choosing an assumption about the shape of the answer.
Underfitting	Too simple. Bad on training data and on new data.
Overfitting	Memorised. Great on training data, bad on new data. Spot it by the gap between train and test scores.
Gradient descent	Feel the slope, step downhill, repeat. new dial = old dial – step size × slope.
Learning rate	The step size. Too small crawls; too big bounces; far too big explodes to nan silently.
Local minimum	A small dip that is not the true bottom. The walker can get stuck.
Perceptron	One neuron: multiply, add, yes/no. Draws only a straight line. 1958.
XOR	Four dots no straight line separates. The perceptron cannot learn it. 1969.
Hidden layer	Moves the dots so a straight line works. 1986.
Activation (“bend”)	Needed after every neuron; without it, many layers = one straight line. Sigmoid, ReLU.
Backpropagation	Works out each weight’s share of the blame, from the output backwards. The “Blame” step.
Why 2012	Data (ImageNet 2009) + compute (GPUs 2007) for a 1986 algorithm.
Five lines	zero_grad · forward · loss · backward · step.

7.1 Twelve words, in plain English

model	a formula with dials
parameter / weight	a dial the computer turns
hyperparameter	a setting you choose before training
loss	one number saying how wrong the guess was; bigger is worse
gradient	the slope: how the loss changes if you turn one dial a little

learning rate	how big a step to take downhill
epoch	one full trip through the training data (one time round the loop)
training set / test set	the dots the model studies / the dots it never sees until the end
generalise	do well on new dots, not just the ones it studied
hidden layer	neurons between the input and the answer; they move the dots
activation function	the bend after each neuron
backpropagation	passing the blame backwards so every dial knows which way to turn

8 Twenty practice questions, with answers

COVERS Quiz preparation

TIME 20 minutes

IN BRIEF Cover the right-hand column, answer, uncover. Every question here is one of the eight quiz topics in disguise.

#	Question	Answer
1	Name the four steps of the training loop, in order.	Guess (forward), Check (loss), Blame (backward), Nudge (update).
2	Which step is the only one that changes the dials?	Nudge (the update / optimiser step).
3	Is the learning rate a parameter or a hyperparameter? Why?	Hyperparameter: you choose it before training; the computer does not learn it.
4	What are the three choices you make before training?	The shape of the answer (model), how to score a mistake (loss), how to search (optimiser).
5	What shape of boundary can logistic regression draw?	Only a straight line (the answer is squashed to a probability, but the boundary is still a line).
6	Why do decision-tree boundaries look like staircases?	Each question is about one feature at a time, so each split is a vertical or horizontal line.
7	What does an SVM look for that plain logistic regression does not?	The line with the widest empty corridor (margin) between the two classes.
8	A model scores 1.000 on training and 0.85 on test. What is wrong?	It has overfitted (memorised the training data). The gap of 0.15 is the tell-tale sign.
9	A model scores 0.60 on training and 0.58 on test. What is wrong?	It is underfitting: too simple to capture the pattern. Small gap, but low everywhere.
10	Why must the test set be data the model has never seen?	Otherwise you are measuring memory, not learning; the score would not predict performance on new data.
11	Say gradient descent in words.	Feel the slope at the current dial settings, take a small step downhill, repeat.
12	Write the gradient-descent update in words.	new dial = old dial – step size × slope.
13	What happens if the learning rate is too small? Too big? Far too big?	Too small: crawls, runs out of time. Too big: bounces over the bottom. Far too big: explodes to nan, with no error message.
14	What is a local minimum?	A small dip on the loss surface that is not the true bottom; the walker feels flat ground and stops there.
15	Describe the perceptron in one sentence.	One neuron: multiply the inputs by weights, add them, and output yes if the total is above a threshold.
16	Why can a perceptron never learn XOR?	XOR's four points cannot be separated by any straight line, and a perceptron can only draw a straight line.

17	What does a hidden layer do that solves XOR?	It moves (re-represents) the four points so that a straight line does separate them.
18	Why is an activation function (“bend”) needed between layers?	Without it, stacked linear layers multiply out to a single linear layer, so depth adds nothing.
19	Backpropagation in one sentence.	The method for working out how much each weight is to blame for the loss, starting from the output and working backwards.
20	List the five PyTorch lines and match each to the loop.	<code>zero_grad</code> (wipe), <code>model(X)</code> (guess), <code>loss_fn</code> (check), <code>loss.backward()</code> (blame), <code>optimiser.step()</code> (nudge).

If you can answer all twenty without looking, you are ready for the Week 3 part of the quiz. The full booklet has 58 more questions in Chapter 21, for anyone who wants more practice; they cover the same eight topics at greater depth.